

A checker for schedule-dependent work amplification in Hydro programs

The problem

A distributed program does work in response to its input, and that work falls into three regimes. In the first, the amount of work is a function of the input alone: a heartbeat sends one message per timer tick to each peer, a transitive closure derives every path in the graph it was given, and no delivery schedule can make either do more. In the second, the program contains a mechanism that reacts to the lateness or absence of a message, and that mechanism is doing the job it was designed for. A client re-sends a request because a packet was dropped, a peer repeats a gossip message until the acknowledgement that was lost arrives, a follower starts an election because the leader crashed, a cache fetches again because an entry was evicted. The extra work in this regime is the intended price of tolerating faults, and it is bounded because the fault ends: the packet is re-sent once, the election elects a leader, the cache is refilled. In the third regime the same mechanism fires, but what triggered it was not a fault. A message was late because the process that should have sent it was overloaded, or because the network was congested, and the program cannot tell "late because lost" from "late because slow". Its response to the first case, sending again, is exactly wrong for the second, because it adds load to the thing that is already too slow, which makes the next message later still, which fires the mechanism again. When that feedback is strong enough, a transient delay pushes the system into a state where it spends its whole capacity on redundant work, completes nothing, and does not recover after the delay is gone.

The mechanisms are intentional and the hazard is their misfiring, which is the standard framing of metastable failure. We call the general property schedule-dependent work amplification and the self-sustaining case metastability. A program that has such a mechanism is not therefore wrong; a program without one is not therefore well designed, since a heartbeat or a transitive closure would be useless in the face of loss. The question the checker answers is whether a program contains a mechanism of the second kind at all, and where in the program it reads lateness, so that the engineer can judge whether the deployment can be driven into the third regime.

This property is hard to test for. The schedules that expose it are rare under random exploration, because they require the same edge to be delayed for many consecutive steps while everything else proceeds normally, and the probability of that under independent random choices falls geometrically with the length of the delay. The property also depends on timing in a way that fixed tests cannot probe: a load test fixes a delivery schedule and varies the load, a unit test fixes both, and neither asks whether some other schedule would have made the same load cost more. What is needed is a way to search over schedules while holding the input fixed, and a way to measure work that does not depend on knowing what the program is for.

The idea, and why it works

The checker rests on five claims, each argued below and each demonstrated on one program. The program is a request-response service with client-side retry. The client forwards each application request to the server with an id, keeps a table of requests awaiting a response, re-sends any request unanswered within 40 of its clock ticks, and gives up after three sends. The server keeps arriving requests in a first-in, first-out queue and serves at most five per tick. The workload throughout is two requests per round against that capacity of five, so under an undisturbed schedule there is no backlog and no re-send ever fires.

The simulator's decisions are an adversarial network's moves

Hydro is a dataflow language for distributed programs whose compiler also produces a deterministic simulator. The simulator cannot control or replay real time, so a program must take its timers as input streams of tick events in order to be checked, and in simulation the harness, not the operating system, decides when each tick event arrives. This is a requirement the checker places on the program and not a property of the language; a Hydro program is free to read the clock or run a real periodic timer, but then the simulator cannot explore what happens when that timer is late. Each process advances in discrete atomic steps (Hydro calls one such

step a tick, and the block of dataflow that runs inside it a `sliced!` block). A step fires only when at least one of its inputs has an item waiting; a step with nothing waiting does not run at all. That is why timers must be inputs, because a step with only internal state would never fire, and it is why holding the only input of a step stops the step altogether rather than asking it a question, which is what the report means when it says a hold was applied by parking the tick.

The simulator makes several kinds of choice, and every one of them is a question put to a pluggable source of answers. Wherever messages wait to enter a step, it decides how many of the waiting messages are delivered into this step and which stay waiting, and for a stream whose order the program does not rely on, which ones and in what order (this point is a `use::batch` call in the program, and we call it a batch decision point). Wherever a step reads a piece of state that is updated by another step, it decides how old a version of that state the step sees (this point is a `use::snapshot` call, and we call it a snapshot decision point); it can show an older version but never one newer than exists. When several steps or processes are ready to run, it decides which runs next. Wherever the program asserts that an unordered stream may be treated as ordered, it decides the interleaving. Where a top-level fold consumes its inputs outside any step, it decides which of the buffered inputs to release and in what order. And when a program opts in to fault injection, it decides whether a member crashes at a send boundary and which of that member's staged messages still get delivered, and when a change of cluster membership becomes visible to each member. Under random or exhaustive exploration all of these choices vary.

The checker varies two of them and fixes the rest. It varies how many waiting messages one batch decision point admits, always as a prefix of what is waiting, with no reordering, and it varies which version one snapshot decision point observes. Every other choice, including which step runs next, every interleaving, and every crash and membership decision, follows the fixed reference schedule described below throughout a check. That is the scope of a benign verdict: it speaks to delays imposed at one point at a time, and not to reorderings, to crashes, or to delays at several points together.

The two decisions the checker varies are exactly what an adversarial network controls when it delays traffic. A network can delay a message or a batch of messages by any amount and then deliver it, and a network can cause a process to act on a stale view of another process's state by delaying the update that would have refreshed it. It cannot invent messages, alter their contents, or change what a process does with what it receives. The simulator's decisions have the same reach: a batch decision point can only delay what passes through it, a snapshot decision point can only show an older version than the newest, and the program's logic runs unchanged on whatever it is given. A schedule of these decisions is therefore a complete description of one adversarial delay pattern, and searching over such schedules is searching over adversaries of that kind. In the retry service the relevant decision points are the server's admission of requests, the client's admission of responses, the client's admission of its own clock ticks, and a few bookkeeping points that report metrics; holding the client's response point for a while is what a network does when it delays the server's replies.

Delaying one point at a time isolates its effect

The checker fixes every decision except one. It first runs the program once under a deterministic reference schedule in which every batch decision point admits everything that is waiting, every snapshot decision point sees the newest version, and steps run in a fixed round-robin order (we call this the prompt schedule). That run establishes the baseline counts and discovers every decision point that ever has something waiting. Then, for each discovered decision point and each hold length k in a fixed sequence, it runs the program again on the same input with that one point held for k consecutive rounds starting from round 1, while every other decision still follows the prompt schedule. The sequence is 1, 2, 4, 8 and so on, doubling up to and including a hold that lasts to the end of the run, which is the schedule in which that point's items are never delivered at all. The checker does not ask the user which delays to try, because a user who knew the program's timescales would not need the checker, and a list of delays that stopped short of a program's timeout would read benign with nothing to say so; the only quantity the user sets is the length of the run, which bounds the longest delay the checker can impose and is printed with every verdict. A held batch point delivers nothing during the hold and everything at once when it ends, which is what a delayed network edge looks like from inside the program. A held snapshot point keeps showing its step the version it saw when the hold began and jumps to the newest version when the hold ends, which is what a delayed state update looks like.

Because the simulator is deterministic and the input is fixed, the held run and the baseline run are identical up to the round at which the hold begins, and every difference between them after that round is caused by the hold. Because only one point is held and the reference schedule elsewhere is the most prompt one possible, nothing in the held run is delayed except what the held point delays and what the program does in consequence. This is what makes the comparison an experiment rather than an observation: the difference between the two runs is the effect of one delay on one edge, with no other perturbation to confound it.

Making a hold of any length a single decision is also what makes the search reach the schedules that matter. A uniform random scheduler decides independently at each step whether a waiting item is released, so the chance that one item survives forty consecutive decisions is on the order of one in a trillion, and a program whose timeout is forty ticks is out of reach of that search entirely. Holding a named point for a chosen number of rounds reaches a forty-round delay as cheaply as a five-round one.

Records admitted and messages sent cannot rise under a delay unless the program derived them

Work is measured passively inside the simulator, with no instrumentation of the program and no knowledge of what its messages mean. Two counts are read over the whole run. The first is the number of records admitted through batch decision points, summed across the program. The second is the number of network messages each process or cluster member sends, kept separately per sender.

Both counts have the property that a delay alone cannot raise them. Over a whole run, a batch point's admitted total is the number of records that arrived at it minus the number still waiting at the end. A hold changes when those records are admitted, in fewer and larger groups, and it may push some past the end of the run, which lowers the total; it does not change how many arrived. The only way the total rises is that more records arrived, and records arrive only because some step upstream produced them. The same holds for messages: a message is sent when the sender's dataflow emits it, and delaying the sender's inputs cannot make its logic emit more unless the logic itself reacts to the delay. A positive difference between the held run and the baseline run on the same input is therefore, by construction, work that the program derived in response to being delayed, which is what redundant work means here. The measure needs no notion of a request, a completion, or a retry; it needs only the fact that delay conserves records.

The retry service shows the count doing exactly this. Holding the client's response point for 1, 2, 4, 8, 16, 32, 64, 128 and 256 rounds produces, in extra messages sent by the client against the baseline, the curve 0 0 0 0 0 48 272 784. Every number in it can be derived by hand. A reply delayed by less than the 40-tick timeout needs no re-send, so holds of 1 through 32 add nothing. Beyond 40, each of the two requests per round issued during the excess is re-sent once, which is two extra client messages per round: a hold of 64 exceeds the timeout by 24 rounds and costs 2 times 24, which is 48. At 128, 88 rounds exceed the timeout and 48 of them exceed twice the timeout, so their requests are re-sent a second time, for 2 times 88 plus 2 times 48, which is 272; at 256 the same arithmetic gives 2 times 216 plus 2 times 176, which is 784. The checker's count and the hand derivation agree to the message, and the count was produced without the checker knowing that the program has requests, replies, or a timeout. Read in admitted records rather than messages the same curve is 0 0 0 0 0 240 1360 3920, because each re-sent request and its reply pass through five decision points in all, and the report shows both. At the two longest holds, 512 rounds and the whole run, the message curve jumps to 3603 and then 3756, and the reason is the second mechanism in the program: from the fortieth round of a hold the client is issuing two first sends and two re-sends per round, and from the eightieth two more, so the server receives six requests per round against a capacity of five and builds a backlog of about one request per round for as long as the hold lasts. When a 512-round hold ends that backlog is more than four hundred requests deep, the server needs well over a hundred rounds to work it off at three net per round, every request that waits in it longer than forty ticks is re-sent again, and the re-sends lengthen the queue, so the storm outlasts the hold and the count approaches its ceiling of two re-sends for every request in the run.

The point that reacts to the shortest delay is where the reaction begins

The verdict is hazardous if any hold of any discovered decision point raises either count above the baseline, and benign otherwise. Messages are read per sender because a cluster total can fall while one member's traffic rises, as when a leader stops sending heartbeats while its followers start sending vote requests. Admitted records are read in total because within one process a reaction can replace one record with another, as when a completion becomes an abandonment, without adding work. The rule uses only the existence of extra work. The shape of the curve, whether the extra work grows with the length of the delay or steps once and stays flat, is reported for the reader and is not a criterion the checker applies.

A hazardous verdict does not say that the program is wrong. It says that the program contains a mechanism that reacts to lateness by doing more work, which is the second regime described at the start of this document, and it names the point where that mechanism reads lateness. Whether a deployment can be driven from that regime into the third, where the reaction feeds itself, depends on capacities, timeouts, and load that the checker does not know, and that judgment belongs to the engineer. What the checker contributes is the existence of the mechanism and its location, established from the program's behaviour alone.

Attributing the extra work to the held point is sound for the following reason. Extra work appears in a held run only if the program's logic, somewhere, turns the lateness of the held records into new records. The held point is the only place lateness was introduced, so the reaction is to the records that pass through that point. Holding a point the program does not react to produces zero at every length, however long the hold, because the records are the same and merely later. Every point with positive extra work is therefore an edge whose lateness the program reacts to, and the report carries all of them with their curves.

Among those points the checker names one as the location, and it ranks them by the shortest hold length at which extra work first appeared, then, among points tied on that length, by the larger extra work at that length, and only as a final tie-break by the largest extra work anywhere on the curve. The point that reacts to the least delay is where the program's response to lateness begins, and that is the fact a reader wants first: it names the edge on which the program tolerates the least lateness. The largest extra work over the whole curve is a poor primary criterion under this sequence of holds, because the last hold lasts to the end of the run, and under a permanent hold on any edge of a resend loop the sender exhausts its resends on every request, so several edges of the same loop reach one ceiling and the largest extra work no longer distinguishes them. The report also lists which other decision points admitted more records under the winning hold, which names the path the extra records took.

In the retry service both edges of the loop first react at a hold of 64 rounds, the first length in the sequence beyond the 40-tick timeout, and the server's request arrivals point adds more extra work there than the client's response point, 415 admitted records against 240, so the checker names the arrivals point, at the line of the server's `use::batch` on its request stream, as the location. The mechanism explains why arrivals react harder. Requests held at the server time out at the client, their re-sends queue behind the held originals, and when the hold ends the server receives all of them at once, so its queueing delay stays above the client's timeout for many rounds after the hold has ended and the re-send storm feeds itself for a while before draining. The report lists the client's response point and the metrics points as the places the extra records crossed. The clock point and the metrics points, held for any length, produce zero, because the program does nothing in response to their lateness except wait. The same program with the retry limit set to one send, so that a timed-out request is abandoned rather than re-sent, produces zero at every hold length on every point, and the verdict is benign: the only edge whose lateness the program reacted to has had its reaction removed.

Extra work that grows with the delay is the signature of feedback

The curve over hold lengths carries more than the verdict. A program that pays a fixed cost for a delay of any length, say one re-send per delayed request however late the reply, produces a step: zero below the threshold and a constant above it. A program whose reaction to delay produces further delay produces a curve that keeps rising with the hold, because each round of excess lateness generates work that itself lengthens the lateness of what follows. The response-point curve of the retry service rises linearly past the timeout and then doubles its slope at twice the timeout, as second re-sends begin, which is the arithmetic of a fixed rule

applied to a growing set of late requests. The arrivals-point curve rises faster than that arithmetic predicts, because the re-sends it provokes lengthen the server's queue and thereby delay the requests behind them, which is the feedback loop that makes retry storms self-sustaining under load. Two shapes appear elsewhere in the corpus described below: a plateau, where the program caps its own reaction at a fixed rate and the storm continues at that rate for the rest of the run, and a rise followed by a fall proportional to the rounds remaining, which is what a program looks like once a hold has tipped it into a state it does not leave. Every length in the sequence is run for every point, with no early stop at the first extra work, so that the report shows this shape and not only its sign; the shape informs the reader and plays no part in the verdict.

What the user provides and what comes back

The user provides three things. The first is the program, written in ordinary Hydro dataflow with its timers as stream parameters, which is how any Hydro program must be written to be simulated at all; a deployment wires periodic timers into the same parameters. The second is a closure that sends one round of a steady workload into the program's inputs, including one tick event to each timer; the checker calls it once per round and then runs the simulation until nothing more can happen before calling it again. In practice the closure is generated from the function's signature by the attribute described under "Running the checker" below, and the user states only the per-round rates. The workload should be the program's ordinary operating point with no burst, because the question the checker answers is whether some schedule makes that same input cost more. The third is the length of the run in rounds, which we call the horizon, with a default of 1000. The horizon is the only setting, and it is not a tuning knob in the sense the hold lengths would have been: it bounds the longest delay the checker can impose, so a benign verdict is the statement that no reaction to a delay of up to the horizon was found at any decision point, and the report says so in those words. The hold lengths themselves are fixed by the doubling sequence, and every hold begins at round 1, after one round of ordinary operation has created the decision points and their state; the program's reaction depends on how long a delay lasts and not on when it starts.

What comes back is a report. It contains the verdict, the baseline counts, and one curve per discovered decision point giving the extra admitted records and the extra messages from each sender at every hold length. For a hazardous program it also contains a location: the decision point that reacted to the shortest delay, given as a source file and line, together with the hold length at which its reaction first appeared, what rose there (messages from which sender, or admitted records) and by how much, its largest extra work over the whole curve, and which other decision points admitted more records under that hold. The line is that of the program's own `use::batch` or `use::snapshot` expression, recorded while the `sliced!` macro that introduces a step expands, so the location names one operator; when two decision points share a line, the report adds each one's position within its step. As each decision point's holds complete the checker prints that point's curve, so a long run shows its findings as it goes. The simulation is compiled once and reused for every run. At the default horizon a benign program with a handful of decision points is checked in 2 to 30 seconds, and a hazardous one in 15 seconds to several minutes; the cost of checking a hazardous program is mostly the cost of simulating the storm the longest holds provoke, since a backlog of thousands of records is re-read every step for hundreds of rounds. The twenty-two corpus configurations described next take about twenty minutes of test time together at the default horizon, of which the delta gossip program with re-sends accounts for more than eight and the two lease-renewal configurations for four, and about a minute and a half at a horizon of 240 rounds.

The report for the running example

The report below is what the checker printed for the retry service with three attempts, at the default horizon, exactly as a user would see it.

```
amplification check: 1000 rounds of workload, every hold begins at round 1
horizon: 999 rounds (the longest delay imposed; a hold of that length lasts to the end of the run)
run time: 27.5 s including compilation
unheld run: 13000 records admitted at decision points, 4000 network messages sent, 8 decision points ha
```

A decision point is named by the file and line of its own use::batch or use::snapshot expression, then :

hold length (rounds)	1	2	4	8	16	32	64	128	256	512
src/cluster/rpc_retry.rs:190, batch of (usize, ()): no extra work at any hold length										
src/cluster/rpc_retry.rs:192, batch of (usize, u64)										
extra records admitted	0	0	0	0	0	0	0	475	12644	10000
extra messages, busiest sender	0	0	0	0	0	0	0	95	3150	30000
first reacted at a hold of 128 rounds with 475 extra; largest extra work 12644 in admitted records										
src/cluster/rpc_retry.rs:193, batch of Response<u64>										
extra records admitted	0	0	0	0	0	0	240	1360	3920	16000
extra messages, busiest sender	0	0	0	0	0	0	48	272	784	30000
first reacted at a hold of 64 rounds with 240 extra; largest extra work 16243 in admitted records										
src/cluster/rpc_retry.rs:296, batch of Request<u64>										
extra records admitted	0	0	0	0	0	0	415	15742	14462	11000
extra messages, busiest sender	0	0	0	0	0	0	83	3756	3756	30000
first reacted at a hold of 64 rounds with 415 extra; largest extra work 15742 in admitted records (1)										
src/cluster/rpc_retry.rs:332, batch of Request<u64>: no extra work at any hold length										
src/cluster/rpc_retry.rs:333, snapshot of usize: no extra work at any hold length										
src/cluster/rpc_retry.rs:358, batch of Completion: no extra work at any hold length (held by parking the)										
src/cluster/rpc_retry.rs:359, batch of Request<u64>: no extra work at any hold length										

verdict: hazardous

decision point: src/cluster/rpc_retry.rs:296, batch of Request<u64>
first reacted at a hold of 64 rounds, with 415 extra admitted records there
largest extra work over the curve: 15742 in admitted records
decision points that admitted more records under that hold, at its peak:
src/cluster/rpc_retry.rs:193, batch of Response<u64> +2357
src/cluster/rpc_retry.rs:296, batch of Request<u64> +3756
src/cluster/rpc_retry.rs:332, batch of Request<u64> +2357
src/cluster/rpc_retry.rs:359, batch of Request<u64> +3756
src/cluster/rpc_retry.rs:360, batch of (u64, u32) +3756
src/cluster/rpc_retry.rs:361, batch of u64 +1387

The first four lines say what was run: 1000 rounds of the steady workload, every hold beginning at round 1, so the longest possible delay is 999 rounds; the whole check took 28 seconds; and the undisturbed run admitted 13000 records at decision points and sent 4000 messages, which is one request and one reply for each of the 2000 requests. The sentence after them explains how decision points are named: by the file and line of the program's own use::batch or use::snapshot expression, so that a reader can open the file at that line and find the operator. The row headed hold length (rounds) gives the eleven hold lengths that every curve's numbers line up under.

The eight decision points are the program's steps' inputs. Lines 190, 192 and 193 are the client's step: line 190 admits the client's clock, line 192 admits the new requests the workload sends in, and line 193 admits replies arriving from the server. Line 296 is the server's step's single input, the requests arriving from the client. Lines 332 and 333 belong to a metrics step on the server that counts processed requests and reads the backlog depth through a snapshot, and lines 358 and 359 belong to a metrics step on the client that counts completions and sends.

The client's reply point, line 193, carries the curve derived by hand earlier in this document. Its second row, extra messages from the busiest sender, reads 48, 272 and 784 at holds of 64, 128 and 256 rounds: two requests per round re-sent once for every round the hold exceeds the forty-tick timeout, and re-sent again for every round it exceeds twice the timeout. Its first row, extra records admitted, is five times larger at each of those lengths, 240, 1360 and 3920, because a re-sent request and its reply are admitted at five decision points on their way through the program (the server's arrivals, the client's replies, and three metrics inputs) while they are one message per hop; the two rows measure the same re-sends at two places, and the report

shows both. At 512 rounds the message row jumps to 3603, for the reason given earlier: the re-sends made during a long hold overload the server, whose queue then outlasts the hold. At 999 rounds the record row is lower than at 512 even though the message row is higher, because a hold to the end of the run pushes some records past the end where they are never admitted.

The client's new-request point, line 192, reacts only from 128 rounds, when the dump of 256 held requests reaching the server at once builds a queue whose delay exceeds the timeout, and its 999-round entry is negative because requests held to the end of the run are never sent at all. The server's arrivals point, line 296, first reacts at the same 64 rounds as the reply point but with more extra work there, 415 records and 83 messages against 240 and 48, and by 128 rounds its message row has already reached 3756, the ceiling of two re-sends for every request in the run: the released requests take the server tens of rounds to drain at three net per round, every one that waits longer than forty ticks is re-sent into the same queue, and the storm never ends. The annotation on that point says the hold worked by parking the server's step, since the step has no other input and cannot run while its only input is held. The two metrics steps and the client's clock show no extra work at any length, because the program does nothing about their lateness except wait.

The verdict block names the server's arrivals point as the location, because it is one of the two points that react at the shortest hold and adds more extra work there, and it lists the decision points that admitted more records under that hold at its peak, which is the path the re-sent requests took: the client's reply input, the server's arrivals, and the metrics inputs.

The same program with the retry limit set to one attempt produces this report.

```
amplification check: 1000 rounds of workload, every hold begins at round 1
horizon: 999 rounds (the longest delay imposed; a hold of that length lasts to the end of the run)
run time: 26.2 s including compilation
unheld run: 13000 records admitted at decision points, 4000 network messages sent, 8 decision points had
```

A decision point is named by the file and line of its own use::batch or use::snapshot expression, then .

hold length (rounds)	1	2	4	8	16	32	64	128	256	512
src/cluster/rpc_retry.rs:190, batch of (usize, ()): no extra work at any hold length										
src/cluster/rpc_retry.rs:192, batch of (usize, u64): no extra work at any hold length										
src/cluster/rpc_retry.rs:193, batch of Response<u64>: no extra work at any hold length										
src/cluster/rpc_retry.rs:296, batch of Request<u64>: no extra work at any hold length (held by parking the server's step)										
src/cluster/rpc_retry.rs:332, batch of Request<u64>: no extra work at any hold length										
src/cluster/rpc_retry.rs:333, snapshot of usize: no extra work at any hold length										
src/cluster/rpc_retry.rs:358, batch of Completion: no extra work at any hold length (held by parking the server's step)										
src/cluster/rpc_retry.rs:359, batch of Request<u64>: no extra work at any hold length										

verdict: benign

no reaction to a delay of up to 999 rounds was found at any decision point

Every decision point shows no extra work at any hold length, including a hold of 999 rounds on the client's reply input, under which every request in the run is eventually abandoned rather than re-sent, and the verdict is benign, stated as a fact about the horizon.

Evaluation

The checker was evaluated against a corpus of twelve small Hydro programs written for the purpose by authors who were given a specification of the property and instructed to know nothing about how it would be detected. The programs cover request-response retry with and without exponential backoff, the same service with a bounded and an unbounded server queue, a read-through cache with a time-to-live with and without coalescing of concurrent misses, delta gossip with and without acknowledgement and re-send, full-state gossip of a grow-only set, heartbeat-timeout leader election, lease renewal against a single server, work-queue rebalancing on stale length reports, a log store whose compaction runs on leftover budget, a

pure heartbeat, and a semi-naive transitive closure. Most programs carry a parameter that changes the mechanism under study, which gives twenty-two configurations in all.

Each configuration carries a label with a stated basis. A configuration is labeled hazardous by exhibited collapse when a test drives it, on the fixed deterministic schedule, through a bounded burst of extra load and shows in the tail of the run, long after the burst has ended, that redundant work persists: a backlog that keeps growing, completions at zero, or a storm of messages still running. For the retry service a sixty-round burst leaves the server backlog growing from 1038 to 1237 over the last two hundred rounds with zero completions against a baseline of four hundred. A configuration is labeled benign by assurance argument when its author wrote down why total work is a closed-form function of the input under every schedule and checked that closed form under a few hundred random schedules. The benign configurations are benign because they contain no mechanism that reacts to lateness, not because they are better designed: the retry service with one attempt abandons a late request instead of re-sending it, the heartbeat and the transitive closure would be useless in the face of loss, and the coalescing cache tolerates a slow origin only because it never has more than one fetch outstanding per key. Ten configurations are hazardous by exhibited collapse and seven are benign by assurance argument. Five more have neither basis: they carry the mechanism of a collapsing configuration together with a tuning parameter such as backoff, a queue bound, or a cooldown that kept the test's burst from collapsing them, and since a tuning parameter can only move the threshold at which a mechanism runs away, the checker is expected to find amplification in them without a label asserting that it must. One program, the log store, is excluded from the scorecard, because its author modeled the cost of a read as an integer inside a record rather than as records moved, so no work actually increases in it and there is nothing for a record count to see.

On the twenty scored configurations the checker agrees with all sixteen ground-truth labels and all four expectations, reading nothing but the simulator's own counts, and the verdicts are the same at a horizon of 240 rounds and at the default of 1000. In every hazardous case the decision point it names is an edge of the loop the program's mechanism runs through. For the three retry-based services it names the server's request arrivals, which react first and hardest to delay. For lease renewal it names the point where acknowledgements arrive, and for rebalancing the point where tasks arrive at a worker. For the cache it names the origin's clock, since a one-round-slower origin is enough to let more lookups miss, before the point where fills arrive reacts at two rounds. For the election it names the followers' timer: a pause of every member's clock followed by the dump of the withheld ticks makes every follower's timeout fire at once, at a hold of 8 rounds, while the point where messages enter a follower's step reacts at 16. For delta gossip it names the members' timer for the same reason, since a re-send is due when an acknowledgement is older than the timeout as judged by the member's own clock, so a paused clock makes every outstanding delta look overdue at once at a hold of 2 rounds, two rounds before the acknowledgement point reacts. Under the earlier fixed grid the cache, the staggered election and the gossip program named the fill, inbox and acknowledgement points instead; those points still react in every case and the report shows their curves, but under the doubling sequence the timer edge reacts to a shorter delay. The count that rose in the election is one follower's outgoing messages, by 3948 at the default horizon, while the leader's traffic fell.

The curve shapes described above all occur. Delta gossip with re-send until acknowledged produced, in extra admitted records under a hold on its acknowledgement point for 1, 2, 4, 8, 16, 32, 64, 128, 256, 512 and 999 rounds, 0 0 60 24878 24910 24910 24910 24910 24910 24910 and then a fall to -50 when the hold never ends, a jump and then a plateau, because the program caps re-sends at one per peer per tick. The read-through cache with request dating and no coalescing produced 0 4 28 64 -228 100 10540 10540 10540 10540 6544 under a hold on the point where fills arrive, with the jump where a held fill's age crosses the twenty-tick time-to-live and every fill arrives stale. The benign programs read flat or falling on every decision point, and a hold that lasts to the end of the run reads negative on them, because the held records are never admitted.

This is agreement on the corpus against which the verdict rule was developed. The choice to read messages per sender and admitted records in total was made after a plain total missed the staggered election and a per-point breakdown wrongly flagged the one-attempt retry service, so the twenty agreements measure how well the rule fits the programs it was fitted to, and not how it will fare on programs it has not seen. The sequence of hold lengths and the location rule were then changed with the corpus as the only check that verdicts survived, so they too were shaped with these programs in view.

A second evaluation was therefore run on programs the checker's developers had not seen. A separate session with no history was given only the specification of the property and the repository, and it wrote seven further programs: a batching sink that retains items until acknowledged, a replicated log catch-up protocol, two-phase commit with participant timeouts, a work queue with a visibility timeout, a bounded publish-subscribe fan-out, credit-based flow control, and a token-bucket rate limiter. It labeled four of them hazardous by exhibited collapse and three benign by assurance argument. The checker, with its rule frozen, was then run on all seven, and its verdicts were recorded before anyone on the checker side read the labels. It agreed on all seven, produced no false hazardous verdict on the three benign programs, and on each hazardous program named the point at which the author's own description places the mechanism: the sink's arrivals of flushed copies, the leader's arrivals of fetch requests, the participant's service timer, and the worker's arrivals of deliveries. Two qualifications apply. The operator who wired the programs saw test names that implied their labels before running the checker, which could not affect the checker's output but did weaken the operator's independence. And all four hazardous programs amplify by the same mechanism the corpus does, a timed re-send whose redundant work crosses a network edge, while the three benign ones never send anything twice, so a hazard of any other shape has not yet been put to the checker. The reports for all seven are in the appendix.

What the checker cannot do

The verdict is a statement about the workload the user supplied, and it can depend on that workload; the log store with a compaction reserve is not amplified by any schedule of one write per four reads and is amplified under three writes per two reads, so a program with a tuning parameter can pass at one operating point and fail at another, and the checker reports only the point it was given. Work that a program performs as arithmetic inside an operator, rather than as records that move, is invisible, because the counts see records entering steps and messages crossing the network and nothing that happens to a number inside a record or to records flowing between operators within one step. Decision points are held one at a time, from round 1, for the lengths in the doubling sequence, so a hazard that requires two edges to be delayed together, or a delay that must begin at a particular moment, is outside the search, and a reaction that is largest at a length between two entries of the sequence is seen only at the entries; the staggered election's inbox reacts most at a hold near 20 rounds and the sequence sees it at 16 and 32, where it is small. A benign verdict is bounded by the horizon: it says that no reaction to a delay of up to the horizon was found, and a program whose reaction begins beyond that is reported benign with the horizon stated. The location rule ranks by the shortest provoking delay, so a point that reacts slightly to a short delay outranks one that reacts strongly to a longer delay; the report carries every point's curve so a reader can see both. A program with no batch or snapshot decision point offers the simulator nothing to hold and is reported benign on that basis alone, as the pure heartbeat is. Snapshot decision points are held but their reads are not counted as work, because a step reads its snapshot exactly once every time it runs regardless of freshness, and no program in the corpus makes its sends depend on a snapshot, so the snapshot hold has not yet met a case that could exercise it.

What a hazardous verdict does and does not tell you

The checker does not apply load. Its perturbation is a stall on one edge for k rounds, with k running as far as the end of the run, and nothing inside the program can shorten that stall; a fast server or a short queue changes what the program does when the stall ends, not how long it lasts. The checker therefore answers "can any delivery schedule make this program do work its input did not require", and not "does this configuration survive an overload it inflicts on itself", which is a question for a load test. Nor can the checker tell a partition from an overload. A stall of 64 rounds on a link is a partition, and a client that re-sends through a partition is the retry mechanism doing its designed job; the checker cannot know whether the stall stands for a lost packet or for a drowning server. It reports that the mechanism exists and where it reads lateness, and the judgment about whether a deployment can turn that mechanism against itself belongs to the engineer.

The bounded-queue program in the corpus makes the distinction concrete. Its server drops queued requests beyond a bound and answers each with an immediate rejection. A bound below the product of the client's timeout and the server's capacity guarantees that a queued request is served before the client's timer fires

and that an unqueued one is rejected at once, so under load the server can never be the reason a reply is late; the witness's burst test confirms that with the bound at 100 the configuration recovers, with its queue empty from round 250 and no request served twice in eight hundred rounds, where the unbounded version collapses. The checker calls the bounded configuration hazardous all the same, and would at any bound. The bound caps how long the server takes, not how long the network takes. Once a stall exceeds the 40-tick timeout the client re-sends whatever the bound is, and when the stall ends the accumulated requests arrive in a number that exceeds any finite bound, so the server rejects most of them, the client backs off and re-sends, and rejections are messages too. In the report the arrivals point first reacts at a hold of 64 rounds, exactly as in the unbounded program; shrinking the bound changes the shape of the curves and not the verdict. In general, a bound on a queue guarantees only that the queue is not the source of the delay, and a retry mechanism reacts to lateness from any source and adds load to whichever component is already the bottleneck.

Each of the following situations leaves the service's queue empty or short and produces a retry storm anyway, and each corresponds to a stall the checker imposes. A garbage-collection or scheduling pause stops the server from reading its socket; nothing enters its queue, every request in flight crosses the timeout at about the same moment, and when the process resumes it finds the originals and their re-sends waiting together. That is a stall on the arrivals edge, and it is what the report describes as a hold applied by parking the tick. Congestion on the reply path, such as a saturated uplink or a proxy that buffers responses, delays replies the server has already produced; the server's queue stays short, the client re-sends, and the duplicated replies add to the congestion that caused the re-send. That is a stall on the replies edge. A slow dependency behind the service makes the reply latency equal to the dependency's latency while the service's intake stays within its bound, and each re-send opens another request against the dependency, which is where the load lands. Slow name resolution or connection setup can consume most of the timeout before a request reaches the service at all, so the retry repeats the expensive part of the exchange for a request the queue never saw.

The bound does accomplish something, and the memory argument says what. A feedback loop needs memory, meaning state that keeps delay above the timeout after the perturbation that started the loop has ended. In the plain retry service that memory is the server's backlog, which re-sends lengthen and which delays the requests behind them. Bounding it means the server forgets a stall within bound-over-capacity ticks, so the re-sends a stall provokes are a transient that drains rather than a steady state that sustains itself. That is why the bounded configuration is hazardous by mechanism and recovers under the burst, and both statements are true of it at once. The bound removes one place where the loop can store its history and cannot remove the others: the client's table of scheduled re-sends, the buffers of a congested link, and a dependency's queue behind the service are all memory, and the last is memory the service cannot see, let alone bound. A pause is not memory, so a bounded queue turns the pause scenario into a clean transient; congestion and a backed-up dependency are memory elsewhere, and there the loop can persist while the server's queue is empty the whole time.

Governance on the client side is the other place to intervene, and it exposes a gap in the checker's present rule. A retry budget refilled by time caps re-sends at a rate; under a stall of k rounds a fixed fraction of the timed-out requests still re-send, so extra work still grows with k , only with a smaller slope, and whether that slope is safe depends on the workload. Such a program stays hazardous. A budget refilled only by successful replies is different in kind: when replies stop, refills stop, re-sends stop, and the extra work a stall can provoke is bounded by the size of the bucket however long the stall lasts, so the curve rises once and goes flat. The mechanism turns itself off when it would be harmful, and by the definition this document uses, more work the longer delivery is delayed, that program is benign even though it contains a retry. The checker's present rule says hazardous on any extra work and would flag it; the definition would not. Until this example every program with a reaction also grew, so rule and definition agreed on every configuration tried. The rule should read the curve: extra work that keeps rising across the doublings is a loop, and extra work that steps up once and holds is a bounded reaction, which deserves a verdict of its own that reports its size. That change has not been made. A witness implementing a success-refilled budget, a hybrid budget refilled by both time and successes, and a circuit breaker is being written independently by an author who has not seen the checker, so that the rule can be tested against it before it changes.

What the simulator already had and what this work added

The checker is built on Hydro's existing simulator and adds to it rather than replacing any part of it. Before this work the simulator already exposed every scheduling decision as a question to a pluggable source of answers. A `use::batch` in a program compiles to a hook (`StreamHook` in `hydro_lang/src/sim/runtime.rs`) that asks, each time its step might run, how many of its waiting items to admit and, for unordered streams, which ones; a `use::snapshot` compiles to a hook (`SingletonHook`) that asks which queued version of its state the step should observe. The answers come from a `bolero Driver`, and the simulator's `fuzz` and `exhaustive` entry points supply random and exhaustive drivers. The simulator also has hooks for the choices the checker does not vary: which ready step runs next, the interleaving of streams the program asserts to be ordered (`StreamOrderHook`, `MergeOrderedHook`, `PartiallyOrderedStreamHook` and their keyed and top-level variants), the release of inputs to top-level folds (`TopLevelFoldHook`), and, when a program opts in, crashes and membership changes (`CrashHook`, `MembershipHook`); under a check all of these answer as the reference schedule dictates. The scheduler, the step loop, the network delivery into a receiver's top-level channel, and the `quiesce` fixpoint were all in place. The deterministic reference schedule (`prompt_schedule.rs`, 114 lines, a driver that admits everything and observes the newest version) and the `run_with_driver` entry point that runs a simulation once under a caller-supplied driver (26 lines in `flow.rs` and 31 in `compiled.rs`) were ported onto this branch from an earlier, abandoned attempt at the same problem before the checker was written, so they are additions in the sense that the main branch did not have them, though the checker did not originate them.

This work added the following, in about 1,750 lines, almost all of them under `hydro_lang/src/sim/`. The hold driver (`hold_one_hook.rs`, 582 lines) is a `Driver` that answers the existing questions with "admit nothing" for one chosen batch hook or "observe the version you saw last" for one chosen snapshot hook, and answers every other question as the reference schedule would; it also records which hooks ever had something waiting, which is how decision points are discovered. Hook identity and kind (113 lines added to `runtime.rs`) let each hook report its source location, its item type, whether it is a batch or a snapshot, and how many items it is about to release, and the scheduler now publishes, around each question, which hook is asking and whether it is being forced to decide (part of 108 lines changed in `compiled.rs`). The source location a hook reports is the line of the program's own `use::batch` or `use::snapshot` expression. That needed a change outside the simulator, because the simulator positions operators by a runtime backtrace and `rustc` collapses the debug-info location of code expanded from an external macro to the macro's call site, so for a program in another crate the backtrace of every decision point inside a `sliced!` block named the block. A new `span_location!` macro in the `copy_span` crate (74 lines) reads the position of the user's tokens while `sliced!` expands, `sliced!` passes it to a new field on the operator's backtrace record (65 lines in `compile/ir/backtrace.rs`, 10 in the macro), and the simulator prefers that position when it has one (part of 32 lines in `builder.rs`). Nothing else reads the field, so backtraces used elsewhere are unchanged. A one-clause change to the scheduler's test of whether a step can run (`SimTick::can_run` in `compiled.rs`) makes a step whose only waiting input sits behind a held hook park instead of being forced to admit something, which is what lets a hold outlive one round; with no hold set the clause is always true and the test is unchanged. The snapshot pin and catch-up rules live in the hold driver: a held snapshot keeps its last version, and on release the first decision skips to the newest queued version so the effect is a delay rather than a lag. The passive work counters (`work_counts.rs`, 239 lines, plus 29 lines in `builder.rs`) count records admitted per hook per cluster member, and network messages per sending member by placing the deployment path's existing `_network_metrics()` pass-through into the simulator's send pipelines; they are off unless a caller enables them. The `check` function, its configuration and its report (`amplification.rs`) run the experiment described above and rank the results. Every addition is inert unless a hold is set or counting is enabled, and the simulator's own test suite passes unchanged.

Running the checker

A user does not write the wiring described above by hand. The attribute `#[amplification_check(...)]`, exported from `hydro_lang::sim::amplification`, is placed on the Hydro function and generates the harness from the function's signature. This is the running example as it stands in the repository:

```

#[amplification_check(
    name = three_attempts,
    T = u64,
    workload(requests = 2),
    policy = RetryPolicy { timeout_ticks: 40, max_attempts: 3 },
    server_config = ServerConfig { max_per_tick: 5, service_time: Duration::ZERO },
)]
#[amplification_check(
    name = one_attempt,
    T = u64,
    workload(requests = 2),
    policy = RetryPolicy { timeout_ticks: 40, max_attempts: 1 },
    server_config = ServerConfig { max_per_tick: 5, service_time: Duration::ZERO },
)]
pub fn rpc_with_retries<'a, T>(
    client: &Process<'a, Client>,
    server: &Process<'a, Server>,
    requests: Stream<T, Process<'a, Client>, Unbounded, TotalOrder, ExactlyOnce>,
    client_clock: Stream<(), Process<'a, Client>, Unbounded>,
    client_report_tick: Stream<(), Process<'a, Client>, Unbounded>,
    server_report_tick: Stream<(), Process<'a, Server>, Unbounded>,
    policy: RetryPolicy,
    server_config: ServerConfig,
) -> RpcOutputs<'a, T>

```

The macro classifies every parameter by its type. A `&Process<'a, X>` or `&Cluster<'a, X>` is a location, which the harness creates; a cluster also needs its size, given by the parameter's name as `cluster = 5`. A `Stream<T, Loc, Unbounded, ...>` is an input, which the harness creates as a simulation input on the matching location and feeds every round; a stream whose item type is `()` is treated the same way, which is exactly what a timer needs, so timers require nothing in the attribute. Anything else is a plain value that must be given by name, and every generic type parameter must be given a concrete type. The struct the function returns derives `SimOutputs`, a one-line addition that lets the harness attach every returned stream to a simulation output so the dataflow is not pruned; a function returning a single stream, a keyed stream, or a tuple of streams needs nothing.

What the user encodes therefore falls into three kinds. The attribute itself says that the function is to be checked, and under what name. The type for `T` and the values for `policy` and `server_config` are what any test would need to call the function at all, and the macro cannot invent them. The `workload(...)` clause is different in kind: it states the operating point, here two requests per round, and it is kept as a separate clause so a reader can see which line is a judgment rather than a fact of the signature. Every stream not named in it receives one value per round. A list, as in `workload(requests = [1, 4, 16])`, runs the check once per rate and reports each verdict, so a verdict that depends on the rate is shown rather than hidden behind one number. When the steady workload is not a rate, for example a cache whose lookups cycle over ten hot keys, the clause `round = |round, inputs| { ... }` replaces the generated workload with a closure that is called once per round and sends into `inputs`, which has one sender per stream parameter. Data inputs draw their values from the trait `InputValue`, which gives sequential integers, unit values, and the like, and which a program with its own input type implements in a few lines. A `horizon = 240` clause sets a shorter run for a first look.

Each attribute expands to a test named `amplification_check_<function>_<name>`, so `cargo test` runs it like any other. The script `cargo-check-amplification` at the repository root runs every such test in a crate, or those whose name contains a substring, and prints a summary table from the lines the tests record under `target/amplification/`; the full report of every check is left under `target/amplification/reports/`. This is the invocation and the table for the running example:

```
$ ./cargo-check-amplification hydro_test rpc_with_retries
```

<code>function</code>	<code>configuration</code>	<code>workload</code>
<code>rpc_with_retries</code>	<code>one_attempt</code>	<code>requests=2, client_clock=1, client_report_tick=1</code>
<code>rpc_with_retries</code>	<code>three_attempts</code>	<code>requests=2, client_clock=1, client_report_tick=1</code>

The verdicts, locations, first-reaction lengths, and extra work are those of the hand-wired tests, and every reacting curve is identical. The line of the location differs from the appendix because the attributes now sit above the function in the source file. One difference in the generated harness is worth knowing: it feeds the two metrics-tick streams that the hand-wired test left silent, since it feeds every stream, so the report shows ten decision points rather than eight and a larger baseline of admitted records; the two additional points show no extra work.

Every program in the corpus and every program of the second evaluation now carries the attribute, twenty-eight configurations in all, and `./cargo-check-amplification hydro_test` runs them together; one of them, the batch flusher from the second evaluation, is marked to be skipped by default because its check takes about twenty minutes, and `--include-ignored` runs it. Their summary table is at the end of the appendix. Two programs could not take the attribute: the full-state gossip in `hydro_std` returns a `Singleton`, which the harness has no way to sink without a step of its own, and its crate does not enable the simulator outside its tests; and nothing else. The compaction store needed one four-line `InputValue` implementation for its operation type, giving one put per four gets, and the cache, the rebalancing workers, the transitive closure, and the token bucket use `round` closures because their steady workloads are not rates.

Where the code is

The checker is the function `hydro_lang::sim::amplification::check`, which takes a simulation, a `CheckConfig`, and a per-round workload closure and returns a `Report`. The attribute and the derive are the proc-macro crate `hydro_amplification_macro`, re-exported from `hydro_lang::sim::amplification`; the runtime pieces the generated harness relies on (`InputValue`, `SimOutputs`, the results recorder) are in `hydro_lang::sim::amplification_harness`; the runner is the script `cargo-check-amplification` at the repository root. The schedule that holds one decision point lives in `hydro_lang::sim::hold_one_hook`, the passive counts in `hydro_lang::sim::work_counts`, and the reference schedule in `hydro_lang::sim::prompt_schedule`. The corpus programs and their labeling tests are under `hydro_test/src/cluster/witnesses/`, with the retry service one directory up in `hydro_test/src/cluster/rpc_retry.rs`, and the tests that run the checker against every corpus configuration and assert the recorded verdict and location are in `hydro_test/src/cluster/witnesses/checker_experiments.rs`. The seven programs of the second evaluation are under `hydro_test/src/cluster/witnesses/blind/`, the tests that run the checker on them are in `hydro_test/src/cluster/witnesses/blind_check.rs`, and the record of that evaluation, with the verdicts as they were written down before the labels were read, is `design_docs/2026-09_blind_results.md`.

Appendix: every report

The reports below are the checker's printed output for every configuration it was run on, at the default horizon of 1000 rounds, captured from one run of the validation tests. A report of twenty-five lines or fewer is reproduced in full. A longer report is abbreviated: its header, the hold-length row, the curves of the decision points that reacted, and its verdict block are kept verbatim, and the one-line entries for decision points that showed no extra work at any hold length are replaced by a sentence giving their number, as is the explanatory sentence that every report repeats. The first two reports are also shown in the running example above.

`rpc_retry`, three attempts (the running example)

```
amplification check: 1000 rounds of workload, every hold begins at round 1
horizon: 999 rounds (the longest delay imposed; a hold of that length lasts to the end of the run)
run time: 27.5 s including compilation
unheld run: 13000 records admitted at decision points, 4000 network messages sent, 8 decision points had
```

hold length (rounds)	1	2	4	8	16	32	64	128	256	!
----------------------	---	---	---	---	----	----	----	-----	-----	---

```

src/cluster/rpc_retry.rs:192, batch of (usize, u64)
  extra records admitted      0      0      0      0      0      0      0      475 12644 10
  extra messages, busiest sender 0      0      0      0      0      0      0      95 3150 3
  first reacted at a hold of 128 rounds with 475 extra; largest extra work 12644 in admitted records
src/cluster/rpc_retry.rs:193, batch of Response<u64>
  extra records admitted      0      0      0      0      0      0      240 1360 3920 16
  extra messages, busiest sender 0      0      0      0      0      0      48 272 784 3
  first reacted at a hold of 64 rounds with 240 extra; largest extra work 16243 in admitted records
src/cluster/rpc_retry.rs:296, batch of Request<u64>
  extra records admitted      0      0      0      0      0      0      415 15742 14462 11
  extra messages, busiest sender 0      0      0      0      0      0      83 3756 3756 3
  first reacted at a hold of 64 rounds with 415 extra; largest extra work 15742 in admitted records (1

```

verdict: hazardous

```

decision point: src/cluster/rpc_retry.rs:296, batch of Request<u64>
first reacted at a hold of 64 rounds, with 415 extra admitted records there
largest extra work over the curve: 15742 in admitted records
decision points that admitted more records under that hold, at its peak:
  src/cluster/rpc_retry.rs:193, batch of Response<u64> +2357
  src/cluster/rpc_retry.rs:296, batch of Request<u64> +3756
  src/cluster/rpc_retry.rs:332, batch of Request<u64> +2357
  src/cluster/rpc_retry.rs:359, batch of Request<u64> +3756
  src/cluster/rpc_retry.rs:360, batch of (u64, u32) +3756
  src/cluster/rpc_retry.rs:361, batch of u64 +1387

```

This report is abbreviated: five decision points that showed no extra work at any hold length are omitted, along with the explanatory sentence.

rpc_retry, one attempt

```

amplification check: 1000 rounds of workload, every hold begins at round 1
horizon: 999 rounds (the longest delay imposed; a hold of that length lasts to the end of the run)
run time: 26.2 s including compilation
unheld run: 13000 records admitted at decision points, 4000 network messages sent, 8 decision points ha

```

A decision point is named by the file and line of its own use::batch or use::snapshot expression, then :

```

hold length (rounds)      1      2      4      8     16     32     64    128    256
src/cluster/rpc_retry.rs:190, batch of (usize, ()): no extra work at any hold length
src/cluster/rpc_retry.rs:192, batch of (usize, u64): no extra work at any hold length
src/cluster/rpc_retry.rs:193, batch of Response<u64>: no extra work at any hold length
src/cluster/rpc_retry.rs:296, batch of Request<u64>: no extra work at any hold length (held by parking
src/cluster/rpc_retry.rs:332, batch of Request<u64>: no extra work at any hold length
src/cluster/rpc_retry.rs:333, snapshot of usize: no extra work at any hold length
src/cluster/rpc_retry.rs:358, batch of Completion: no extra work at any hold length (held by parking th
src/cluster/rpc_retry.rs:359, batch of Request<u64>: no extra work at any hold length

```

verdict: benign

no reaction to a delay of up to 999 rounds was found at any decision point

backoff_retry, backoff on

```

amplification check: 1000 rounds of workload, every hold begins at round 1
horizon: 999 rounds (the longest delay imposed; a hold of that length lasts to the end of the run)
run time: 16.3 s including compilation

```

unheld run: 7000 records admitted at decision points, 4000 network messages sent, 4 decision points had

hold length (rounds)	1	2	4	8	16	32	64	128	256	512
src/cluster/witnesses/backoff_retry.rs:175, batch of (usize, u64)										
extra records admitted	0	0	0	0	0	0	0	34	2278	3135
extra messages, busiest sender	0	0	0	0	0	0	0	17	1139	2014
first reacted at a hold of 128 rounds with 34 extra; largest extra work 3135 in admitted records										
src/cluster/witnesses/backoff_retry.rs:176, batch of Response<u64>										
extra records admitted	0	0	0	0	0	0	42	308	1294	4014
extra messages, busiest sender	0	0	0	0	0	0	21	154	647	2014
first reacted at a hold of 64 rounds with 42 extra; largest extra work 4014 in admitted records										
src/cluster/witnesses/backoff_retry.rs:263, batch of Request<u64>										
extra records admitted	0	0	0	0	0	0	62	1226	5354	4014
extra messages, busiest sender	0	0	0	0	0	0	31	613	3637	3135
first reacted at a hold of 64 rounds with 62 extra; largest extra work 5354 in admitted records (held)										

verdict: hazardous

decision point: src/cluster/witnesses/backoff_retry.rs:263, batch of Request<u64>
first reacted at a hold of 64 rounds, with 62 extra admitted records there
largest extra work over the curve: 5354 in admitted records
decision points that admitted more records under that hold, at its peak:
src/cluster/witnesses/backoff_retry.rs:176, batch of Response<u64> +1717
src/cluster/witnesses/backoff_retry.rs:263, batch of Request<u64> +3637

This report is abbreviated: one decision point that showed no extra work at any hold length is omitted, along with the explanatory sentence.

backoff_retry, backoff off

amplification check: 1000 rounds of workload, every hold begins at round 1

horizon: 999 rounds (the longest delay imposed; a hold of that length lasts to the end of the run)

run time: 15.0 s including compilation

unheld run: 7000 records admitted at decision points, 4000 network messages sent, 4 decision points had

hold length (rounds)	1	2	4	8	16	32	64	128	256	512
src/cluster/witnesses/backoff_retry.rs:175, batch of (usize, u64)										
extra records admitted	0	0	0	0	0	0	0	190	4867	3135
extra messages, busiest sender	0	0	0	0	0	0	0	95	3150	3135
first reacted at a hold of 128 rounds with 190 extra; largest extra work 4867 in admitted records										
src/cluster/witnesses/backoff_retry.rs:176, batch of Response<u64>										
extra records admitted	0	0	0	0	0	0	96	544	1568	6440
extra messages, busiest sender	0	0	0	0	0	0	48	272	784	3135
first reacted at a hold of 64 rounds with 96 extra; largest extra work 6440 in admitted records										
src/cluster/witnesses/backoff_retry.rs:263, batch of Request<u64>										
extra records admitted	0	0	0	0	0	0	166	6113	5473	4014
extra messages, busiest sender	0	0	0	0	0	0	83	3756	3756	3135
first reacted at a hold of 64 rounds with 166 extra; largest extra work 6113 in admitted records (held)										

verdict: hazardous

decision point: src/cluster/witnesses/backoff_retry.rs:263, batch of Request<u64>
first reacted at a hold of 64 rounds, with 166 extra admitted records there
largest extra work over the curve: 6113 in admitted records
decision points that admitted more records under that hold, at its peak:
src/cluster/witnesses/backoff_retry.rs:176, batch of Response<u64> +2357
src/cluster/witnesses/backoff_retry.rs:263, batch of Request<u64> +3756

This report is abbreviated: one decision point that showed no extra work at any hold length is omitted, along with the explanatory sentence.

bounded_queue_rejection, queue bounded at 100

amplification check: 1000 rounds of workload, every hold begins at round 1

horizon: 999 rounds (the longest delay imposed; a hold of that length lasts to the end of the run)

run time: 13.1 s including compilation

unheld run: 7000 records admitted at decision points, 4000 network messages sent, 4 decision points had

hold length (rounds)	1	2	4	8	16	32	64	128	256	512
src/cluster/witnesses/bounded_queue_rejection.rs:177, batch of (usize, u64)										
extra records admitted	0	0	0	0	0	0	50	772	1796	3844
extra messages, busiest sender	0	0	0	0	0	0	25	386	898	1965
first reacted at a hold of 64 rounds with 50 extra; largest extra work 3844 in admitted records										
src/cluster/witnesses/bounded_queue_rejection.rs:178, batch of Reply										
extra records admitted	0	0	0	0	0	0	96	544	1634	3756
extra messages, busiest sender	0	0	0	0	0	0	48	272	817	1965
first reacted at a hold of 64 rounds with 96 extra; largest extra work 3756 in sends from Process(1)										
src/cluster/witnesses/bounded_queue_rejection.rs:308, batch of Request										
extra records admitted	0	0	0	0	0	0	244	858	1882	3930
extra messages, busiest sender	0	0	0	0	0	0	122	429	941	1965
first reacted at a hold of 64 rounds with 244 extra; largest extra work 3930 in admitted records (h										

verdict: hazardous

decision point: src/cluster/witnesses/bounded_queue_rejection.rs:308, batch of Request

first reacted at a hold of 64 rounds, with 244 extra admitted records there

largest extra work over the curve: 3930 in admitted records

decision points that admitted more records under that hold, at its peak:

src/cluster/witnesses/bounded_queue_rejection.rs:178, batch of Reply +1965

src/cluster/witnesses/bounded_queue_rejection.rs:308, batch of Request +1965

This report is abbreviated: one decision point that showed no extra work at any hold length is omitted, along with the explanatory sentence.

bounded_queue_rejection, unbounded queue

amplification check: 1000 rounds of workload, every hold begins at round 1

horizon: 999 rounds (the longest delay imposed; a hold of that length lasts to the end of the run)

run time: 18.2 s including compilation

unheld run: 7000 records admitted at decision points, 4000 network messages sent, 4 decision points had

hold length (rounds)	1	2	4	8	16	32	64	128	256	512
src/cluster/witnesses/bounded_queue_rejection.rs:177, batch of (usize, u64)										
extra records admitted	0	0	0	0	0	0	0	190	4867	3844
extra messages, busiest sender	0	0	0	0	0	0	0	95	3150	3844
first reacted at a hold of 128 rounds with 190 extra; largest extra work 4867 in admitted records										
src/cluster/witnesses/bounded_queue_rejection.rs:178, batch of Reply										
extra records admitted	0	0	0	0	0	0	96	544	1568	6440
extra messages, busiest sender	0	0	0	0	0	0	48	272	784	3756
first reacted at a hold of 64 rounds with 96 extra; largest extra work 6440 in admitted records										
src/cluster/witnesses/bounded_queue_rejection.rs:308, batch of Request										
extra records admitted	0	0	0	0	0	0	166	6113	5473	4867
extra messages, busiest sender	0	0	0	0	0	0	83	3756	3756	3844
first reacted at a hold of 64 rounds with 166 extra; largest extra work 6113 in admitted records (h										

verdict: hazardous

decision point: src/cluster/witnesses/bounded_queue_rejection.rs:308, batch of Request
first reacted at a hold of 64 rounds, with 166 extra admitted records there
largest extra work over the curve: 6113 in admitted records
decision points that admitted more records under that hold, at its peak:
src/cluster/witnesses/bounded_queue_rejection.rs:178, batch of Reply +2357
src/cluster/witnesses/bounded_queue_rejection.rs:308, batch of Request +3756

This report is abbreviated: one decision point that showed no extra work at any hold length is omitted, along with the explanatory sentence.

cache_thundering_herd, request-dated, no coalescing

amplification check: 1000 rounds of workload, every hold begins at round 1

horizon: 999 rounds (the longest delay imposed; a hold of that length lasts to the end of the run)

run time: 24.3 s including compilation

unheld run: 11400 records admitted at decision points, 1400 network messages sent, 5 decision points ha

hold length (rounds)	1	2	4	8	16	32	64	128	256	512
src/cluster/witnesses/cache_thundering_herd.rs:183, batch of (usize, u32)										
extra records admitted	-200	-196	-192	-176	-100	10332	10202	9946	9434	8000
extra messages, busiest sender	-100	-98	-96	-88	-50	7156	7154	7154	7154	7154
first reacted at a hold of 32 rounds with 10332 extra; largest extra work 10332 in admitted records										
src/cluster/witnesses/cache_thundering_herd.rs:184, batch of Fill										
extra records admitted	0	4	28	64	-228	100	10540	10540	10540	10540
extra messages, busiest sender	0	2	14	32	-114	50	7240	7240	7240	7240
first reacted at a hold of 2 rounds with 4 extra; largest extra work 10540 in admitted records										
src/cluster/witnesses/cache_thundering_herd.rs:299, batch of ()										
extra records admitted	4	16	36	76	-216	188	10540	10540	10540	10540
extra messages, busiest sender	2	8	18	38	-108	94	7240	7240	7240	7240
first reacted at a hold of 1 rounds with 4 extra; largest extra work 10540 in admitted records										
src/cluster/witnesses/cache_thundering_herd.rs:300, batch of Fetch										
extra records admitted	0	4	8	24	-72	10358	10230	9974	9462	8000
extra messages, busiest sender	0	2	4	12	-36	7182	7182	7182	7182	7182
first reacted at a hold of 2 rounds with 4 extra; largest extra work 10358 in admitted records										

verdict: hazardous

decision point: src/cluster/witnesses/cache_thundering_herd.rs:299, batch of ()
first reacted at a hold of 1 rounds, with 4 extra admitted records there
largest extra work over the curve: 10540 in admitted records
decision points that admitted more records under that hold, at its peak:
src/cluster/witnesses/cache_thundering_herd.rs:184, batch of Fill +3300
src/cluster/witnesses/cache_thundering_herd.rs:300, batch of Fetch +7240

This report is abbreviated: one decision point that showed no extra work at any hold length is omitted, along with the explanatory sentence.

cache_thundering_herd, fill-dated, no coalescing

amplification check: 1000 rounds of workload, every hold begins at round 1

horizon: 999 rounds (the longest delay imposed; a hold of that length lasts to the end of the run)

run time: 17.9 s including compilation

unheld run: 11008 records admitted at decision points, 1008 network messages sent, 5 decision points ha

hold length (rounds)	1	2	4	8	16	32	64	128	256	512
----------------------	---	---	---	---	----	----	----	-----	-----	-----

```

src/cluster/witnesses/cache_thundering_herd.rs:182, batch of (usize, ())
  extra records admitted      4      8      8      8      8      8      -32     -92    -212    -4
  extra messages, busiest sender  2      4      4      4      4      4     -16     -46   -106    -5
  first reacted at a hold of 1 rounds with 4 extra; largest extra work 8 in admitted records
src/cluster/witnesses/cache_thundering_herd.rs:183, batch of (usize, u32)
  extra records admitted     -4      0      4     20     96    512     980    2252    3976    53
  extra messages, busiest sender -2      0      2     10     48    256     490    1126    1988    38
  first reacted at a hold of 4 rounds with 4 extra; largest extra work 5308 in admitted records
src/cluster/witnesses/cache_thundering_herd.rs:184, batch of Fill
  extra records admitted      0      8     28     64    148    368     836    1812    3716    73
  extra messages, busiest sender  0      4     14     32     74    184     418     906    1858    37
  first reacted at a hold of 2 rounds with 8 extra; largest extra work 7436 in sends from Process(loc1v1)
src/cluster/witnesses/cache_thundering_herd.rs:299, batch of ()
  extra records admitted      4     20     36     76    164    384     852    1824    3736    73
  extra messages, busiest sender  2     10     18     38     82    192     426     912    1868    37
  first reacted at a hold of 1 rounds with 4 extra; largest extra work 7436 in sends from Process(loc1v1)
src/cluster/witnesses/cache_thundering_herd.rs:300, batch of Fetch
  extra records admitted      0      4      8     24    108    368    1048    1808    3708    53
  extra messages, busiest sender  0      2      4     12     54    184     524     904    1854    37
  first reacted at a hold of 2 rounds with 4 extra; largest extra work 7374 in sends from Process(loc1v1)

verdict: hazardous
  decision point: src/cluster/witnesses/cache_thundering_herd.rs:299, batch of ()
  first reacted at a hold of 1 rounds, with 4 extra messages (sends from Process(loc1v1)) there
  largest extra work over the curve: 7436 in sends from Process(loc1v1)
  decision points that admitted more records under that hold, at its peak:
    src/cluster/witnesses/cache_thundering_herd.rs:184, batch of Fill +3496
    src/cluster/witnesses/cache_thundering_herd.rs:300, batch of Fetch +3746

```

This report is abbreviated: the explanatory sentence is omitted.

cache_thundering_herd, coalescing

```

amplification check: 1000 rounds of workload, every hold begins at round 1
horizon: 999 rounds (the longest delay imposed; a hold of that length lasts to the end of the run)
run time: 14.8 s including compilation
unheld run: 11000 records admitted at decision points, 1000 network messages sent, 5 decision points had

```

A decision point is named by the file and line of its own use::batch or use::snapshot expression, then :

```

hold length (rounds)      1      2      4      8     16     32     64    128    256    512
src/cluster/witnesses/cache_thundering_herd.rs:182, batch of (usize, ()): no extra work at any hold length
src/cluster/witnesses/cache_thundering_herd.rs:183, batch of (usize, u32): no extra work at any hold length
src/cluster/witnesses/cache_thundering_herd.rs:184, batch of Fill: no extra work at any hold length
src/cluster/witnesses/cache_thundering_herd.rs:299, batch of (): no extra work at any hold length
src/cluster/witnesses/cache_thundering_herd.rs:300, batch of Fetch: no extra work at any hold length

```

```

verdict: benign
  no reaction to a delay of up to 999 rounds was found at any decision point

```

compaction_falls_behind, reserve 0 (excluded from the scorecard)

```

amplification check: 1000 rounds of workload, every hold begins at round 1
horizon: 999 rounds (the longest delay imposed; a hold of that length lasts to the end of the run)
run time: 8.2 s including compilation

```

unheld run: 6000 records admitted at decision points, 0 network messages sent, 2 decision points had bu

A decision point is named by the file and line of its own use::batch or use::snapshot expression, then :

hold length (rounds)	1	2	4	8	16	32	64	128	256	!
src/cluster/witnesses/compaction_falls_behind.rs:161, batch of (): no extra work at any hold length										
src/cluster/witnesses/compaction_falls_behind.rs:162, batch of (usize, 0p): no extra work at any hold l										

verdict: benign

no reaction to a delay of up to 999 rounds was found at any decision point

compaction_falls_behind, reserve 8 (excluded from the scorecard)

amplification check: 1000 rounds of workload, every hold begins at round 1

horizon: 999 rounds (the longest delay imposed; a hold of that length lasts to the end of the run)

run time: 5.4 s including compilation

unheld run: 6000 records admitted at decision points, 0 network messages sent, 2 decision points had bu

A decision point is named by the file and line of its own use::batch or use::snapshot expression, then :

hold length (rounds)	1	2	4	8	16	32	64	128	256	!
src/cluster/witnesses/compaction_falls_behind.rs:161, batch of (): no extra work at any hold length										
src/cluster/witnesses/compaction_falls_behind.rs:162, batch of (usize, 0p): no extra work at any hold l										

verdict: benign

no reaction to a delay of up to 999 rounds was found at any decision point

crdt_gossip_load

amplification check: 60 rounds of workload, every hold begins at round 1

horizon: 59 rounds (the longest delay imposed; a hold of that length lasts to the end of the run)

run time: 32.0 s including compilation

unheld run: 180 records admitted at decision points, 720 network messages sent, 3 decision points had bu

A decision point is named by the file and line of its own use::batch or use::snapshot expression, then :

hold length (rounds)	1	2	4	8	16	32	59
hydro_std/src/ec_inference_demos/crdt_gossip.rs:71, snapshot of BTreeSet<u32>: no extra work at any hold							
hydro_std/src/ec_inference_demos/crdt_gossip.rs:75, batch of (): no extra work at any hold length							
src/cluster/witnesses/checker_experiments.rs:2879, snapshot of BTreeSet<u32>: no extra work at any hold							

verdict: benign

no reaction to a delay of up to 59 rounds was found at any decision point

election_stampede, uniform timeouts

amplification check: 1000 rounds of workload, every hold begins at round 1

horizon: 999 rounds (the longest delay imposed; a hold of that length lasts to the end of the run)

run time: 19.8 s including compilation

unheld run: 14000 records admitted at decision points, 4000 network messages sent, 3 decision points ha

A decision point is named by the file and line of its own use::batch or use::snapshot expression, then :

hold length (rounds)	1	2	4	8	16	32	64	128	256	!
src/cluster/witnesses/election_stampede.rs:227, batch of (usize, ())										

extra records admitted	-4	-8	-16	-23	-61	-115	-242	-497	-1010	-1
extra messages, busiest sender	-4	-8	-16	3944	3872	3768	3536	3072	2136	3

first reacted at a hold of 8 rounds with 3944 extra; largest extra work 3944 in sends from Cluster(

src/cluster/witnesses/election_stampede.rs:228, batch of Msg

extra records admitted	0	0	0	2	16	38	68	138	260	8
extra messages, busiest sender	0	0	0	3936	3892	3772	3540	3060	2100	0

first reacted at a hold of 8 rounds with 3936 extra; largest extra work 3936 in sends from Cluster(

src/cluster/witnesses/election_stampede.rs:229, batch of u64

extra records admitted	0	0	0	0	0	4	9	14	9	
extra messages, busiest sender	0	0	0	0	0	3764	3536	3080	2136	3

first reacted at a hold of 32 rounds with 3764 extra; largest extra work 3764 in sends from Cluster

verdict: hazardous

decision point: src/cluster/witnesses/election_stampede.rs:227, batch of (usize, ())

first reacted at a hold of 8 rounds, with 3944 extra messages (sends from Cluster(loc1v1) @1) there
largest extra work over the curve: 3944 in sends from Cluster(loc1v1) @1

election_stampede, timeouts spread by 3

amplification check: 1000 rounds of workload, every hold begins at round 1

horizon: 999 rounds (the longest delay imposed; a hold of that length lasts to the end of the run)

run time: 18.4 s including compilation

unheld run: 14000 records admitted at decision points, 4000 network messages sent, 3 decision points ha

A decision point is named by the file and line of its own use::batch or use::snapshot expression, then :

hold length (rounds)	1	2	4	8	16	32	64	128	256	5
----------------------	---	---	---	---	----	----	----	-----	-----	---

src/cluster/witnesses/election_stampede.rs:227, batch of (usize, ())

extra records admitted	-4	-8	-16	-32	-67	-137	-282	-572	-1154	-2
extra messages, busiest sender	-4	-8	-16	3948	3892	17	24	48	93	1

first reacted at a hold of 8 rounds with 3948 extra; largest extra work 3948 in sends from Cluster(

src/cluster/witnesses/election_stampede.rs:228, batch of Msg

extra records admitted	0	0	0	0	-6	-8	-12	-15	-35	-
extra messages, busiest sender	0	0	0	0	13	21	49	99	200	4

first reacted at a hold of 16 rounds with 13 extra; largest extra work 444 in sends from Cluster(lo

src/cluster/witnesses/election_stampede.rs:229, batch of u64

extra records admitted	0	0	0	0	0	-24	-31	-64	-135	-5
extra messages, busiest sender	0	0	0	0	0	3780	20	41	89	1

first reacted at a hold of 32 rounds with 3780 extra; largest extra work 3780 in sends from Cluster

verdict: hazardous

decision point: src/cluster/witnesses/election_stampede.rs:227, batch of (usize, ())

first reacted at a hold of 8 rounds, with 3948 extra messages (sends from Cluster(loc1v1) @1) there
largest extra work over the curve: 3948 in sends from Cluster(loc1v1) @1

gossip_resend, re-sends on

amplification check: 1000 rounds of workload, every hold begins at round 1

horizon: 999 rounds (the longest delay imposed; a hold of that length lasts to the end of the run)

run time: 496.8 s including compilation

unheld run: 49990 records admitted at decision points, 39990 network messages sent, 4 decision points ha

hold length (rounds)	1	2	4	8	16	32	64	128	256	5
----------------------	---	---	---	---	----	----	----	-----	-----	---

src/cluster/witnesses/gossip_resend.rs:122, batch of (usize, ())

extra records admitted	0	9952	24795	24615	24255	23535	22095	19215	13455	1
------------------------	---	------	-------	-------	-------	-------	-------	-------	-------	---

extra messages, busiest sender	0	1999	4959	4923	4851	4707	4419	3843	2691	3
first reacted at a hold of 2 rounds with 9952 extra; largest extra work 24795 in admitted records										
src/cluster/witnesses/gossip_resend.rs:123, batch of (usize, u64)										
extra records admitted	0	0	14824	24554	24200	23480	22040	19160	13400	1
extra messages, busiest sender	0	0	2977	4913	4842	4698	4410	3834	2682	3
first reacted at a hold of 4 rounds with 14824 extra; largest extra work 24554 in admitted records										
src/cluster/witnesses/gossip_resend.rs:124, batch of Delta										
extra records admitted	0	8	24803	24720	24525	24125	23325	21725	18525	12
extra messages, busiest sender	0	3	4963	4945	4907	4827	4667	4347	3707	2
first reacted at a hold of 2 rounds with 8 extra; largest extra work 24803 in admitted records										
src/cluster/witnesses/gossip_resend.rs:125, batch of Ack										
extra records admitted	0	0	60	24878	24910	24910	24910	24910	24910	24
extra messages, busiest sender	0	0	12	4980	4984	4984	4984	4984	4984	4
first reacted at a hold of 4 rounds with 60 extra; largest extra work 24910 in admitted records										

verdict: hazardous

decision point: src/cluster/witnesses/gossip_resend.rs:122, batch of (usize, ())
first reacted at a hold of 2 rounds, with 9952 extra admitted records there
largest extra work over the curve: 24795 in admitted records
decision points that admitted more records under that hold, at its peak:
src/cluster/witnesses/gossip_resend.rs:124, batch of Delta +19900
src/cluster/witnesses/gossip_resend.rs:125, batch of Ack +4895

This report is abbreviated: the explanatory sentence is omitted.

gossip_resend, re-sends off

amplification check: 1000 rounds of workload, every hold begins at round 1
horizon: 999 rounds (the longest delay imposed; a hold of that length lasts to the end of the run)
run time: 160.0 s including compilation
unheld run: 49990 records admitted at decision points, 39990 network messages sent, 4 decision points ha

A decision point is named by the file and line of its own use::batch or use::snapshot expression, then :

hold length (rounds)	1	2	4	8	16	32	64	128	256	!
src/cluster/witnesses/gossip_resend.rs:122, batch of (usize, ()): no extra work at any hold length										
src/cluster/witnesses/gossip_resend.rs:123, batch of (usize, u64): no extra work at any hold length										
src/cluster/witnesses/gossip_resend.rs:124, batch of Delta: no extra work at any hold length										
src/cluster/witnesses/gossip_resend.rs:125, batch of Ack: no extra work at any hold length										

verdict: benign

no reaction to a delay of up to 999 rounds was found at any decision point

lease_renewal_storm, re-send after 4 ticks

amplification check: 1000 rounds of workload, every hold begins at round 1
horizon: 999 rounds (the longest delay imposed; a hold of that length lasts to the end of the run)
run time: 164.2 s including compilation
unheld run: 24998 records admitted at decision points, 3998 network messages sent, 3 decision points ha

A decision point is named by the file and line of its own use::batch or use::snapshot expression, then :

hold length (rounds)	1	2	4	8	16	32	64	128	256	!
src/cluster/witnesses/lease_renewal_storm.rs:184, batch of (usize, ()): no extra work at any hold length										
src/cluster/witnesses/lease_renewal_storm.rs:185, batch of Ack										

extra records admitted	0	0	4	60	14893	14906	14906	14906	14906	14906	14906
extra messages, busiest sender	0	0	2	30	2984	2984	2984	2984	2984	2984	2984

first reacted at a hold of 4 rounds with 4 extra; largest extra work 14906 in admitted records
src/cluster/witnesses/lease_renewal_storm.rs:267, batch of (MemberId<LeaseClient>, Renewal)

extra records admitted	0	0	2	14807	14819	14743	14589	14269	13629	12900
extra messages, busiest sender	0	0	1	2959	2919	2839	2679	2359	1719	12900

first reacted at a hold of 4 rounds with 2 extra; largest extra work 14819 in admitted records (held)

verdict: hazardous

decision point: src/cluster/witnesses/lease_renewal_storm.rs:185, batch of Ack
first reacted at a hold of 4 rounds, with 4 extra admitted records there
largest extra work over the curve: 14906 in admitted records
decision points that admitted more records under that hold, at its peak:
src/cluster/witnesses/lease_renewal_storm.rs:185, batch of Ack +2984
src/cluster/witnesses/lease_renewal_storm.rs:267, batch of (MemberId<LeaseClient>, Renewal) +11922

lease_renewal_storm, one outstanding renewal

amplification check: 1000 rounds of workload, every hold begins at round 1
horizon: 999 rounds (the longest delay imposed; a hold of that length lasts to the end of the run)
run time: 102.2 s including compilation
unheld run: 24998 records admitted at decision points, 3998 network messages sent, 3 decision points had

A decision point is named by the file and line of its own use::batch or use::snapshot expression, then

hold length (rounds)	1	2	4	8	16	32	64	128	256	512
----------------------	---	---	---	---	----	----	----	-----	-----	-----

src/cluster/witnesses/lease_renewal_storm.rs:184, batch of (usize, ()): no extra work at any hold length
src/cluster/witnesses/lease_renewal_storm.rs:185, batch of Ack: no extra work at any hold length
src/cluster/witnesses/lease_renewal_storm.rs:267, batch of (MemberId<LeaseClient>, Renewal): no extra work

verdict: benign

no reaction to a delay of up to 999 rounds was found at any decision point

pure_heartbeat_load

amplification check: 1000 rounds of workload, every hold begins at round 1
horizon: 999 rounds (the longest delay imposed; a hold of that length lasts to the end of the run)
run time: 2.5 s including compilation
unheld run: 0 records admitted at decision points, 9000 network messages sent, 0 decision points had

A decision point is named by the file and line of its own use::batch or use::snapshot expression, then

hold length (rounds)	1	2	4	8	16	32	64	128	256	512
----------------------	---	---	---	---	----	----	----	-----	-----	-----

verdict: benign

no reaction to a delay of up to 999 rounds was found at any decision point

rebalancing_ping_pong, no cooldown

amplification check: 1000 rounds of workload, every hold begins at round 1
horizon: 999 rounds (the longest delay imposed; a hold of that length lasts to the end of the run)
run time: 14.7 s including compilation
unheld run: 9500 records admitted at decision points, 1000 network messages sent, 4 decision points had

A decision point is named by the file and line of its own use::batch or use::snapshot expression, then

hold length (rounds)	1	2	4	8	16	32	64	128	256	512
src/cluster/witnesses/rebalancing_ping_pong.rs:159, batch of (usize, ()): no extra work at any hold length										
src/cluster/witnesses/rebalancing_ping_pong.rs:160, batch of (>): no extra work at any hold length										
src/cluster/witnesses/rebalancing_ping_pong.rs:161, batch of u64										
extra records admitted	0	0	0	120	251	547	1252	2796	5834	11986
extra messages, busiest sender	0	0	0	65	131	284	639	1422	2948	6000
first reacted at a hold of 8 rounds with 120 extra; largest extra work 11986 in admitted records										
src/cluster/witnesses/rebalancing_ping_pong.rs:163, batch of (MemberId<Worker>, usize): no extra work at any hold length										

verdict: hazardous

decision point: src/cluster/witnesses/rebalancing_ping_pong.rs:161, batch of u64
first reacted at a hold of 8 rounds, with 120 extra admitted records there
largest extra work over the curve: 11986 in admitted records
decision points that admitted more records under that hold, at its peak:
src/cluster/witnesses/rebalancing_ping_pong.rs:162, batch of (MemberId<Worker>, Task) +11986

rebalancing_ping_pong, cooldown 8

amplification check: 1000 rounds of workload, every hold begins at round 1
horizon: 999 rounds (the longest delay imposed; a hold of that length lasts to the end of the run)
run time: 15.0 s including compilation
unheld run: 9500 records admitted at decision points, 1000 network messages sent, 4 decision points had bugs

A decision point is named by the file and line of its own use::batch or use::snapshot expression, then the batch or snapshot expression.

hold length (rounds)	1	2	4	8	16	32	64	128	256	512
src/cluster/witnesses/rebalancing_ping_pong.rs:159, batch of (usize, ()): no extra work at any hold length										
src/cluster/witnesses/rebalancing_ping_pong.rs:160, batch of (>): no extra work at any hold length										
src/cluster/witnesses/rebalancing_ping_pong.rs:161, batch of u64										
extra records admitted	0	0	0	30	70	142	286	574	1150	2302
extra messages, busiest sender	0	0	0	19	37	73	145	289	577	1150
first reacted at a hold of 8 rounds with 30 extra; largest extra work 2302 in admitted records										
src/cluster/witnesses/rebalancing_ping_pong.rs:163, batch of (MemberId<Worker>, usize): no extra work at any hold length										

verdict: hazardous

decision point: src/cluster/witnesses/rebalancing_ping_pong.rs:161, batch of u64
first reacted at a hold of 8 rounds, with 30 extra admitted records there
largest extra work over the curve: 2302 in admitted records
decision points that admitted more records under that hold, at its peak:
src/cluster/witnesses/rebalancing_ping_pong.rs:162, batch of (MemberId<Worker>, Task) +2302

transitive_closure_load

amplification check: 1000 rounds of workload, every hold begins at round 1
horizon: 999 rounds (the longest delay imposed; a hold of that length lasts to the end of the run)
run time: 15.5 s including compilation
unheld run: 1200 records admitted at decision points, 0 network messages sent, 2 decision points had bugs

A decision point is named by the file and line of its own use::batch or use::snapshot expression, then the batch or snapshot expression.

hold length (rounds)	1	2	4	8	16	32	64	128	256	512
src/local/productive_tc.rs:76, batch of Vec<(u32, u32)>: no extra work at any hold length										
src/local/productive_tc.rs:80, batch of (>): no extra work at any hold length (held by parking the tick, then the batch)										

verdict: benign

no reaction to a delay of up to 999 rounds was found at any decision point

blind: batch_flush

amplification check: 1000 rounds of workload, every hold begins at round 1

horizon: 999 rounds (the longest delay imposed; a hold of that length lasts to the end of the run)

run time: 1315.2 s including compilation

unheld run: 8000 records admitted at decision points, 4000 network messages sent, 5 decision points had

hold length (rounds)	1	2	4	8	16	32	64	128	256	512
src/cluster/witnesses/blind/batch_flush.rs:103, batch of (usize, ())										
extra records admitted	0	0	4	52	831418	825620	793572	733116	563318	302
extra messages, busiest sender	0	0	2	26	825554	819884	788092	728148	559374	300
first reacted at a hold of 4 rounds with 4 extra; largest extra work 831418 in admitted records										
src/cluster/witnesses/blind/batch_flush.rs:104, batch of (usize, ())										
extra records admitted	0	0	-8	16	828826	824518	792868	732470	563152	302
extra messages, busiest sender	0	0	-4	8	822960	818780	787386	727500	559206	300
first reacted at a hold of 8 rounds with 16 extra; largest extra work 828826 in admitted records										
src/cluster/witnesses/blind/batch_flush.rs:105, batch of u64										
extra records admitted	0	0	4	814530	856056	869192	885364	893188	902896	930
extra messages, busiest sender	0	0	2	808554	850080	863216	879388	887212	896920	924
first reacted at a hold of 4 rounds with 4 extra; largest extra work 998988 in sends from Process(1)										
src/cluster/witnesses/blind/batch_flush.rs:162, batch of ()										
extra records admitted	0	0	28	168	855568	868898	885398	893038	902824	930
extra messages, busiest sender	0	0	14	84	849576	862906	879406	887046	896832	924
first reacted at a hold of 4 rounds with 28 extra; largest extra work 998988 in sends from Process(1)										
src/cluster/witnesses/blind/batch_flush.rs:163, batch of FlushCopy										
extra records admitted	0	0	60	855714	870396	883372	897516	908576	927870	961
extra messages, busiest sender	0	0	30	849786	864532	877636	892036	903608	923926	959
first reacted at a hold of 4 rounds with 60 extra; largest extra work 998988 in sends from Process(1)										

verdict: hazardous

decision point: src/cluster/witnesses/blind/batch_flush.rs:163, batch of FlushCopy

first reacted at a hold of 4 rounds, with 60 extra messages (sends from Process(loc1v1)) there

largest extra work over the curve: 998988 in sends from Process(loc1v1)

decision points that admitted more records under that hold, at its peak:

src/cluster/witnesses/blind/batch_flush.rs:105, batch of u64 +1896

src/cluster/witnesses/blind/batch_flush.rs:163, batch of FlushCopy +959300

This report is abbreviated: the explanatory sentence is omitted.

blind: log_catchup

amplification check: 1000 rounds of workload, every hold begins at round 1

horizon: 999 rounds (the longest delay imposed; a hold of that length lasts to the end of the run)

run time: 15.7 s including compilation

unheld run: 6161 records admitted at decision points, 2161 network messages sent, 4 decision points had

hold length (rounds)	1	2	4	8	16	32	64	128	256	512
src/cluster/witnesses/blind/log_catchup.rs:127, batch of RecordReply										
extra records admitted	0	0	0	0	78	247	715	1794	8695	8
extra messages, busiest sender	0	0	0	0	72	228	660	1656	7920	7
first reacted at a hold of 16 rounds with 78 extra; largest extra work 8721 in admitted records										
src/cluster/witnesses/blind/log_catchup.rs:219, batch of ()										

extra records admitted	0	0	0	0	91	260	611	1547	8769	8
extra messages, busiest sender	0	0	0	0	84	240	564	1428	7998	7

first reacted at a hold of 16 rounds with 91 extra; largest extra work 8801 in admitted records

src/cluster/witnesses/blind/log_catchup.rs:220, batch of ()

extra records admitted	0	0	0	1	-5	-28	-61	-140	-285	-1
extra messages, busiest sender	0	0	0	1	7	20	47	100	207	4

first reacted at a hold of 8 rounds with 1 extra; largest extra work 825 in sends from Process(loc1v1)

src/cluster/witnesses/blind/log_catchup.rs:221, batch of Fetch

extra records admitted	0	0	0	0	182	8338	8128	7517	6250	3
extra messages, busiest sender	0	0	0	0	168	7678	7358	6718	5438	2

first reacted at a hold of 16 rounds with 182 extra; largest extra work 8338 in admitted records

verdict: hazardous

decision point: src/cluster/witnesses/blind/log_catchup.rs:220, batch of ()

first reacted at a hold of 8 rounds, with 1 extra messages (sends from Process(loc1v1)) there largest extra work over the curve: 825 in sends from Process(loc1v1)

decision points that admitted more records under that hold, at its peak:

src/cluster/witnesses/blind/log_catchup.rs:221, batch of Fetch +1

This report is abbreviated: the explanatory sentence is omitted.

blind: two_phase_commit

amplification check: 1000 rounds of workload, every hold begins at round 1

horizon: 999 rounds (the longest delay imposed; a hold of that length lasts to the end of the run)

run time: 361.1 s including compilation

unheld run: 9000 records admitted at decision points, 6000 network messages sent, 5 decision points had

hold length (rounds)	1	2	4	8	16	32	64	128	256	!
----------------------	---	---	---	---	----	----	----	-----	-----	---

src/cluster/witnesses/blind/two_phase_commit.rs:203, batch of (usize, ())

extra records admitted	-3	-3	-3	170090	182351	185497	182508	172451	138734	83
extra messages, busiest sender	-1	-1	-1	167377	179616	182828	180001	170360	137157	82

first reacted at a hold of 8 rounds with 170090 extra; largest extra work 185497 in admitted records

src/cluster/witnesses/blind/two_phase_commit.rs:204, batch of Vote

extra records admitted	0	0	-3	13	176444	189566	197147	205527	209503	220
extra messages, busiest sender	0	0	-1	7	173675	186733	194278	202648	206522	217

first reacted at a hold of 8 rounds with 13 extra; largest extra work 246504 in sends from Process()

src/cluster/witnesses/blind/two_phase_commit.rs:241, batch of ()

extra records admitted	0	-3	6	181753	190954	197999	205104	211512	218881	231
extra messages, busiest sender	0	-1	3	178984	188179	195270	202507	209159	217166	231

first reacted at a hold of 4 rounds with 6 extra; largest extra work 246504 in sends from Process()

src/cluster/witnesses/blind/two_phase_commit.rs:242, batch of ParticipantMessage

extra records admitted	-3	-3	1	179566	190683	197932	205065	211499	218871	231
extra messages, busiest sender	-1	-1	1	176807	187908	195201	202466	209146	217154	231

first reacted at a hold of 4 rounds with 1 extra; largest extra work 246504 in sends from Process()

verdict: hazardous

decision point: src/cluster/witnesses/blind/two_phase_commit.rs:241, batch of ()

first reacted at a hold of 4 rounds, with 6 extra messages (sends from Process(loc1v1)) there largest extra work over the curve: 246504 in sends from Process(loc1v1)

decision points that admitted more records under that hold, at its peak:

src/cluster/witnesses/blind/two_phase_commit.rs:242, batch of ParticipantMessage +246504

This report is abbreviated: one decision point that showed no extra work at any hold length is omitted, along with the explanatory sentence.

blind: visibility_queue

amplification check: 1000 rounds of workload, every hold begins at round 1

horizon: 999 rounds (the longest delay imposed; a hold of that length lasts to the end of the run)

run time: 211.4 s including compilation

unheld run: 8000 records admitted at decision points, 4000 network messages sent, 5 decision points had

hold length (rounds)	1	2	4	8	16	32	64	128	256	512
src/cluster/witnesses/blind/visibility_queue.rs:115, batch of (usize, u64)										
extra records admitted	0	0	0	0	10	104084	113093	108799	92078	55111
extra messages, busiest sender	0	0	0	0	5	101247	110416	106442	90361	55111
first reacted at a hold of 16 rounds with 10 extra; largest extra work 113093 in admitted records										
src/cluster/witnesses/blind/visibility_queue.rs:119, batch of u64										
extra records admitted	0	0	0	8	56	102028	122432	130996	135871	144111
extra messages, busiest sender	0	0	0	4	28	99055	119459	128023	132898	141111
first reacted at a hold of 8 rounds with 8 extra; largest extra work 165336 in sends from Process(1)										
src/cluster/witnesses/blind/visibility_queue.rs:196, batch of (usize, ())										
extra records admitted	0	0	0	12	64	398	121992	130877	135789	144111
extra messages, busiest sender	0	0	0	6	32	199	118995	127880	132792	141111
first reacted at a hold of 8 rounds with 12 extra; largest extra work 165336 in sends from Process(1)										
src/cluster/witnesses/blind/visibility_queue.rs:200, batch of Delivery										
extra records admitted	0	0	0	16	108269	122714	130526	137260	143294	153111
extra messages, busiest sender	0	0	0	8	105352	119877	127849	134903	141577	152111
first reacted at a hold of 8 rounds with 16 extra; largest extra work 165336 in sends from Process(1)										

verdict: hazardous

decision point: src/cluster/witnesses/blind/visibility_queue.rs:200, batch of Delivery

first reacted at a hold of 8 rounds, with 16 extra messages (sends from Process(loc1v1)) there largest extra work over the curve: 165336 in sends from Process(loc1v1)

decision points that admitted more records under that hold, at its peak:

src/cluster/witnesses/blind/visibility_queue.rs:119, batch of u64 +437

src/cluster/witnesses/blind/visibility_queue.rs:200, batch of Delivery +152818

This report is abbreviated: one decision point that showed no extra work at any hold length is omitted, along with the explanatory sentence.

blind: bounded_fanout

amplification check: 1000 rounds of workload, every hold begins at round 1

horizon: 999 rounds (the longest delay imposed; a hold of that length lasts to the end of the run)

run time: 8.1 s including compilation

unheld run: 8000 records admitted at decision points, 6000 network messages sent, 2 decision points had

A decision point is named by the file and line of its own use::batch or use::snapshot expression, then :

hold length (rounds)	1	2	4	8	16	32	64	128	256	512
src/cluster/witnesses/blind/bounded_fanout.rs:111, batch of Delivery: no extra work at any hold length										
src/cluster/witnesses/blind/bounded_fanout.rs:89, batch of (usize, u64): no extra work at any hold length										

verdict: benign

no reaction to a delay of up to 999 rounds was found at any decision point

blind: credit_flow

amplification check: 1000 rounds of workload, every hold begins at round 1

horizon: 999 rounds (the longest delay imposed; a hold of that length lasts to the end of the run)

run time: 14.5 s including compilation

unheld run: 8000 records admitted at decision points, 4000 network messages sent, 5 decision points had

A decision point is named by the file and line of its own use::batch or use::snapshot expression, then :

hold length (rounds)	1	2	4	8	16	32	64	128	256	!
src/cluster/witnesses/blind/credit_flow.rs:125, batch of (usize, ()): no extra work at any hold length										
src/cluster/witnesses/blind/credit_flow.rs:126, batch of (usize, ()): no extra work at any hold length										
src/cluster/witnesses/blind/credit_flow.rs:127, batch of Ack: no extra work at any hold length										
src/cluster/witnesses/blind/credit_flow.rs:179, batch of (): no extra work at any hold length										
src/cluster/witnesses/blind/credit_flow.rs:180, batch of Job: no extra work at any hold length										

verdict: benign

no reaction to a delay of up to 999 rounds was found at any decision point

blind: token_bucket

amplification check: 1000 rounds of workload, every hold begins at round 1

horizon: 999 rounds (the longest delay imposed; a hold of that length lasts to the end of the run)

run time: 4.4 s including compilation

unheld run: 4000 records admitted at decision points, 3000 network messages sent, 2 decision points had

A decision point is named by the file and line of its own use::batch or use::snapshot expression, then :

hold length (rounds)	1	2	4	8	16	32	64	128	256	!
src/cluster/witnesses/blind/token_bucket.rs:81, batch of (usize, ()): no extra work at any hold length										
src/cluster/witnesses/blind/token_bucket.rs:82, batch of Request: no extra work at any hold length										

verdict: benign

no reaction to a delay of up to 999 rounds was found at any decision point

The summary table for every annotated program

This is the output of ./cargo-check-amplification --threads 4 hydro_test at the default horizon, 27 checks in 636 seconds with the batch flusher skipped. The order is the order in which the tests finished.

function	configuration	workload
pure_heartbeat	default	cluster=3 members, timer=1
rpc_with_backoff	backoff_off	requests=2, client_clock=1
bounded_fanout	default	subscribers=3 members, publications=2
rpc_with_backoff	backoff_on	requests=2, client_clock=1
credit_flow	default	jobs=2, producer_clock=1, consumer_clock=1
replicated_log_catchup	default	appends=2, unrelated_work=0, follower_clock=1
token_bucket	default	round closure
rpc_with_retries	one_attempt	requests=2, client_clock=1, client_report_tick=1
rpc_with_retries	three_attempts	requests=2, client_clock=1, client_report_tick=1
rpc_with_bounded_queue	bounded_100	requests=2, client_clock=1
rpc_with_bounded_queue	unbounded	requests=2, client_clock=1
cache_with_expiry	coalesce	round closure
cache_with_expiry	fill_dated_no_coalesce	round closure
log_with_compaction	reserve_0	ops=5, clock=1
cache_with_expiry	request_dated_no_coalesce	round closure
log_with_compaction	reserve_8	ops=5, clock=1
election	spread_3	cluster=5 members, timer=1, client_requests=1
election	uniform_timeouts	cluster=5 members, timer=1, client_requests=1

visibility_queue	default	submissions=2, broker_clock=1, worker_clock=1
gossip_with_resend	resend_off	cluster=5 members, timer=1, local_updates=1
lease_renewal	one_outstanding	clients=20 members, client_clock=1, server_clock=1
rebalancing_workers	cooldown_8	workers=2 members, round closure
rebalancing_workers	no_cooldown	workers=2 members, round closure
productive_transitive_closure	default	round closure
two_phase_commit	default	transactions=1, coordinator_timer=1, participation_timeout=1
lease_renewal	resend_after_4	clients=20 members, client_clock=1, server_clock=1
gossip_with_resend	resend_on	cluster=5 members, timer=1, local_updates=1